

web-sanitizer-rs — Final Project Report

Requirements, Architecture, Rust Systems-Programming Aspects, Evaluation, and Critical Assessment

September 2026

Table of Contents

Introduction	2
1.1 Context	2
1.2 Goals of this report	3
Requirements Analysis	3
2.1 Problem statement	3
2.2 Functional requirements.....	3
2.3 Non-functional requirements.....	4
2.4 Explicit non-goals.....	5
Architectural Decisions.....	5
3.1 The pipeline.....	5
3.2 Why a policy-driven design.....	6
3.3 Why <i>lol_html</i>	7
3.4 Why the fetch/guard split.....	7
3.5 Module boundaries.....	7
3.6 Mapping onto the suggested architecture.....	8
Rust System-Programming Aspects	9
4.1 Concurrency model	9
4.2 Lifetimes	11
4.3 Error handling.....	12
4.4 <i>unsafe</i> code	13

Experimental Evaluation.....	13
5.1 Correctness.....	13
5.2 Performance.....	15
5.3 Scalability	19
5.4 Resource usage	20
5.5 The adversarial-scenario harness	21
5.6 Summary	21
Critical Assessment: Limitations and Possible Extensions.....	22
6.1 The zip-bomb check trusts declared metadata, not verified decompression.....	22
6.2 MIME-type coverage is deliberately limited to widespread formats.....	22
6.3 Sub-resource policy is uniform across types, not per-type	22
6.4 Active-content rewriting is safe and linear for only four formats	23
6.5 A development-environment limitation: Docker networking on Windows	23
6.6 Possible extensions	24

Introduction

1.1 Context

web-sanitizer-rs (binary name *wsrs*) is a command-line and library tool for sanitising untrusted web content: HTML documents, the sub-resources they reference, and a handful of adjacent formats (SVG, XML, PDF, TIFF, ZIP, images, audio and video).

The tool takes files, directories, or http(s) URLs as input (through API), runs each one through a fixed pipeline, with multiple threads if more than one file is present (and multi-threading is possible), and returns both the sanitised bytes and a structured JSON report describing exactly which rules fired, where, and why.

The main goal of this project is to provide a basic but efficient defence against common malicious attacks, including dangerous html constructs, suspicious URLs, sub-resource manipulation, and other.

Each file is thoroughly sniffed to find the real contents, and enforces customizable rules for each file type.

1.2 Goals of this report

This report documents the design and implementation of `web-sanitizer-rs`, covering:

- the **requirements** the system was built to satisfy, both functional (what it sanitises, and how) non-functional (safety, performance, operability);
- the **architectural decisions** behind the acquire → sniff → route → sanitise → report pipeline, and why the codebase is split the way it is;
- a focused discussion of the **Rust systems-programming** aspects: concurrency model, use of lifetimes, error-handling discipline, and (deliberate) avoidance of `unsafe`;
- an **experimental evaluation** based on the project's test suite and performance benchmarks;
- a **critical assessment** of the current limitations of the system and of the extensions that could be implemented.

Requirements Analysis

2.1 Problem statement

When interacting with web content, a user can be at risk of coming into contact with malicious code. *web-sanitizer-rs* must be the first defence against possible attacks, thus the requirements revolve around these risks:

1. **Active content smuggled through a passive format.**
Scripts, event handlers, dangerous URL schemes, or auto-refreshing frames embedded in HTML; entity-expansion attacks embedded in XML/SVG; polyglot files that are valid under more than one parser.
2. **Resource-exhaustion attacks.**
Zip bombs, deeply nested archives, oversized images, and decompression-ratio attacks that trade a small input for an unbounded amount of work.
3. **Network-side abuse of the sanitiser itself.**
If the tool is allowed to fetch a document's sub-resources, a malicious document can try to reach internal infrastructure, e.g. **SSRF** (Server-Side Request Forgery).

2.2 Functional requirements

#	Requirement
F1	Accept files, directories, <i>--input-list</i> files, and <i>http(s)</i> URLs as input, and process them as an ordered batch.

#	Requirement
F2	Determine the real content type of every input by inspecting its bytes (magic numbers, structural markers), never by trusting a declared <i>Content-Type</i> or file extension alone.
F3	For HTML: remove or rewrite <code><script></code> tags, inline <i>on*</i> event-handler attributes, <i>javascript:/data:</i> URLs, disallowed <code><iframe>/<object>/<embed></code> elements, and <code><meta http-equiv="refresh"></code> redirects, each recorded as a distinct, independently configurable rule.
F4	For URL-bearing attributes (<i>href, src, action, formaction, data, poster</i>): check the host against a configurable block-list and against a set of "protected" domains for homograph/IDN spoofing.
F4b	Detect and neutralise URL-confusion classes: raw IP literal that resolves into a disallowed network (private/link-local/loopback/benchmarking ranges), and credentials embedded in the URL authority used to hide the real host.
F5	For XML/SVG: bound entity expansion to defend against billion-laughs-style attacks.
F6	For compressed/archive inputs (gzip, zip/OOXML): bound the decompression ratio and re-sniff the inflated content recursively, under a depth cap.
F7	Optionally fetch the sub-resources an HTML document references, sanitise them with the same pipeline, and embed them next to the sanitised parent under a joint budget (request count, byte count, depth) and never past depth 1.
F8	Guard every outbound fetch (input URLs and sub-resource URLs) against SSRF: resolve the hostname, check every resolved address against a deny table of private/link-local/loopback ranges, and refuse the connection if any of them match.
F9	Defend against connection-level slow-drip (Slowloris-style) DoS on outbound fetches, not just against an overall time cap.
F10	Produce a structured JSON report for every run: one entry per input, the status it ended in (<i>Clean, Sanitized, Refused, BudgetExceeded, SsrfBlocked, FetchError, ...</i>), and the exact list of actions taken, each with a rule ID, a category, and a source location.
F11	Run under a configurable worker pool for batch throughput, preserving deterministic output naming regardless of completion order.
F12	Load sanitisation policy from a TOML file that overrides a compiled-in default section by section, and fail fast (exit code 2) on unknown keys rather than silently ignoring a typo.

2.3 Non-functional requirements

#	Requirement
N1	Fail closed, not open. A pipeline bug must degrade to a refusal, never to passing unsanitised bytes through.

#	Requirement
N2	Bounded resource usage per input , independent of what the input claims about itself.
N3	No unbounded recursion. Gzip/zip re-entry into <i>route</i> and the sub-resource loop both carry explicit depth caps.
N4	The sanitiser must not become an SSRF vector. Every socket the process opens, for an input URL or a sub-resource, must pass the same address-level check.
N5	Auditability. Every transformation must be individually traceable: rule ID, category, before/after fragment, and byte/line location.
N6	Configuration errors are caught before any input is processed , not discovered mid-batch.
N7	Throughput under batch workloads. Many inputs must be processed concurrently without input-level state leaking across workers.
N8	Determinism of on-disk output naming , independent of which worker finishes first.

2.4 Explicit non-goals

As per requested, these "non-requirements" are a limit to what is in the scope of the project:

- It is **not a general antivirus/content-scanning engine**: it inspects headers and structural risk indicators, not payload semantics.
- It **does not execute scripts in a sandbox**: it merely identifies them and enforces a predetermined action.

Architectural Decisions

3.1 The pipeline

Every input, regardless of source, goes through the same five-stage pipeline, implemented in *src/engine/mod.rs* and *src/engine/route.rs*:

Pipeline stages: Acquire → Sniff → Route → Sanitise → Sub-resource loop → Report

(*Route feeds a gzip/zip payload back into Sniff, and the Sub-resource loop is opt-in*)

- **Acquire** reads bytes from a file, from an in-memory *Bytes* input, or performs the initial HTTP fetch for a URL input, under the input-bytes budget.
- **Sniff** (*src/sniff/mod.rs*) classifies the real MIME type from a table of magic numbers, and compares it against the declared ``Content-Type``. On mismatch, the sniffed type wins and the mismatch itself is recorded as an action.

- **Route** (*src/engine/route.rs*) dispatches on the sniffed type to the right handler:
 - *html::sanitize_html* for HTML;
 - entity-expansion scanning for XML/SVG;
 - header-only dimension checks for images;
 - bounded inflate + re-sniff for gzip;
 - byte-identical pass-through for everything else;
 - This is also where **type-specific budgets** are applied.
- **Sanitise** produces the rewritten bytes plus a list of *SanitisationAction*.
- **Sub-resource loop** (*src/engine/subresource.rs*, opt-in via policy) resolves the references an HTML sanitisation pass collected, fetches them under a joint budget, and recurses the whole pipeline on each one.
- **Report** (*src/report.rs*) assembles a JSON document combining a per-run summary and a per-input report tree (including sub-resources).

3.2 Policy

Policy is the single source of truth for sanitisation rules, actions, and resource limits. It is loaded from compiled-in defaults and can be overridden with TOML. All policy structs use *#[serde(deny_unknown_fields)]*, so configuration typos fail instead of silently weakening protection.

At *Engine::new*, block-lists, protected domains, and SSRF rules are compiled into reusable runtime structures. Invalid block-lists or CIDRs therefore fail before processing begins, and the compiled policy can be shared safely across workers.

Every rule resolves to one of five actions: *Remove*, *Placeholder*, *Rewrite*, *Refuse*, or *Allow*.

Policy section	Controls
<i>HtmlRules</i>	Scripts, event handlers, dangerous URL schemes, frames, meta refresh, and their allow-lists
<i>UrlRules</i>	Host block-lists, protected domains, homograph detection, and URL replacements
<i>Budgets</i>	Input size, processing time, decompression ratio, entity expansions, and image dimensions
<i>FetchPolicy</i>	Connection/read/total timeouts, redirects, response size, and user agent
<i>InputRules</i>	File extensions included during directory walks
<i>SubresourcesRules</i>	Optional recursive fetching, depth/request/byte limits, allowed types, nested ZIP/XML limits, and reference rewriting
<i>SsrfRules</i>	Guard scope, same-origin exemptions, allowed or denied hosts, and allow-cache lifetime

Each section is independently configurable; omitted sections retain their defaults. Format-specific code consumes the compiled policy but does not define policy values itself.

3.3 Why *lol_html*

sanitize_html is built on *lol_html*, a streaming HTML tokenizer/rewriter, instead of a DOM-based library. This is mostly for 2 reasons:

1. **Bounded memory under adversarial input.** A streaming pass never materialises the whole document tree, so memory use tracks input size linearly rather than depending on document structure (e.g. deeply nested elements cannot blow up memory the way they could with some DOM parsers).
2. **Correctness across read boundaries.** A `<script>` tag split across a chunk boundary is still caught, because the tokenizer's state persists across chunks; a naive line-by-line or chunk-by-chunk regex approach would not have this property.

The cost is architectural too: because there is no tree, *sanitize_html* cannot ask cross-element questions. Instead the design uses a single flat handler (*element!("***", ...)*) invoked once per element, with a shared *Ctx* accumulating actions, refusal state, and collected references as a side effect of each per-element callback.

Input is fed to the rewriter as raw bytes, never via *rewrite_str*, specifically so that malformed or non-UTF-8 input degrades to a refusal instead of a panic.

3.4 Why the *fetch/guard* split

src/fetch/mod.rs is the only module in the crate that opens a socket. The SSRF guard (*src/fetch/guard/*) is layered *underneath* the HTTP client rather than as a check the caller remembers to run before calling it:

Fetch/guard layering: Engine / Sub-resource loop → *HttpFetcher* → *ureq* Agent → *GuardedResolver* → *Guard* → { *IpDenyTable*, *ResolveCache* }

(Every caller goes through the same HttpFetcher; the SSRF check is embedded inside the resolver the HTTP agent was built with, so no caller can bypass it.)

GuardedResolver implements *ureq*'s *Resolver* trait, so the SSRF check happens as part of DNS resolution itself, inside the transport the HTTP agent was constructed with.

Every caller funnels through the same *Agent*, so there is structurally no code path that can "forget" to guard a request. Opening a socket already checks the host.

The *FetchContext*/*FetchOrigin* types additionally let the guard apply different scopes (*InputCli*, *InputServer*, *Subresource*) and a same-origin exemption for a sub-resource served by its parent's already-vetted endpoint, without any of that policy logic leaking into the fetcher itself.

3.5 Module boundaries

Module	Responsibility	Notably does <i>not</i> do
input	Turn CLI args / <i>--input-list</i> into an ordered <i>Vec<InputSource></i> ; walk	Any I/O on the input's <i>content</i> .

Module	Responsibility	Notably does <i>not</i> do
	directories, filter by extension, refuse symlinks that escape the tree root.	
sniff	Bytes → <i>MimeType</i> , reconciled against declared type.	Sanitisation.
engine::route	Type → handler dispatch, type-specific budget enforcement.	Knowing <i>how</i> to sanitise any given type.
html	HTML/attribute-level rule application, URL-attribute delegation to <i>urlcheck</i> .	Fetching; SSRF policy.
urlcheck	Block-list / homoglyph / malformed-host classification for a single URL, with its own verdict cache.	Sanitising HTML around the URL.
fetch + fetch::guard	The only socket-opening code path; SSRF enforcement.	Sanitisation, sniffing.
scan	Structural risk scanning (entity expansion, zip/decompress ratio, PDF/TIFF active-content indicators).	HTML rule enforcement.
policy	Declarative configuration schema, compiled artefacts (<i>BlockSet</i> , <i>SkeletonSet</i>).	Applying the policy itself.
report	The JSON report schema and its assembly helpers.	Deciding <i>what</i> goes in a report.
engine	Orchestration: wires every module above into the five-stage pipeline, owns the worker pool.	Format-specific logic.

This pipeline, having *engine* as the module that orchestrates all the different format-specific modules (without having any knowledge itself) allows for each module to be tested in isolation, to check how effective it is in its own domain.

3.6 Mapping onto the suggested architecture

The module separation follows also the proposed architecture in the project specifications:

Suggested layer	Realised as
Input layer (local files, directory trees, remote fetches)	<i>input + fetch/fetch::guard</i>
Parsing layer (safe HTML parser)	<i>html + sniff</i>
Core engine (policy-driven rule)	<i>engine + policy + report</i>

Suggested layer	Realised as
sequencing, scheduler, report aggregation)	
CLI front-end	<i>src/main.rs</i> + <i>src/args.rs</i> , calling only <i>Engine::process/process_batch</i>

The only deviation from the architecture, is that the parsing layer is actually divided in two. *sniff* decides what a byte stream is, before a handler parses and sanitises it.

Rust System-Programming Aspects

This chapter talks about the concurrency model, use of lifetimes, error-handling discipline, and the project's stance on *unsafe*.

4.1 Concurrency model

web-sanitizer-rs uses a **thread-pool-with-channels** model for batch processing, built directly on *std::thread* and *std::sync::mpsc*.

Each input's processing is CPU/IO-bound in short, independent bursts (parse, sanitise, maybe one bounded round of sub-fetches).

4.1.1 The worker pool

Engine::process_batch (*src/engine/mod.rs*) sets up two channels: a job channel (*usize*, *InputSource*) and a result channel (*usize*, *Outcome*), and spawns *jobs.max(1)* OS threads, each looping on:

```
loop {
    let job = { let guard = job_rx.lock().unwrap(); guard.recv() };
    let (index, input) = match job { Ok(pair) => pair, Err(_) => break };
    let outcome = engine.process_indexed(index, input);
    let _ = res_tx.send((index, outcome));
}
```

Three choices are important:

- **The receiving end of the job channel is behind *Arc<Mutex<Receiver>>*.**
mpsc::Receiver is not *Sync*, so it cannot be shared directly across threads. Wrapping it in a *Mutex* turns many workers pulling from one queue into a correct work-stealing queue.
 Since *recv()* blocks and each job is comparatively large-grained, the lock is held only for the instant of receiving a job, not during the work itself.
- **The engine is wrapped in *Arc<Engine>* and cloned per worker.**
Engine itself is *#[derive(Clone)]* over fields that are all *Arc<...>* (*policy*, *blockset*, *skeletonset*, *verdictcache*, and a *dyn Fetcher* behind an *Arc*), so cloning it is cheap and

every worker sees the same compiled policy and the same caches without any data being copied per thread.

- **Index-tagged results restore order deterministically.**

Because workers finish in whatever order the work happens to take, the result channel carries *(index, Outcome)* pairs, and *process_batch* uses the index to name output files and to place each report in its original position.

This is what makes the tool's on-disk output deterministic across runs even though the *scheduling* is not.

4.1.2 Shared, lock-guarded caches

Two caches are shared across the whole worker pool: the URL-verdict cache (*urlcheck::cache::VerdictCache*) and the DNS-resolution verdict cache (*fetch::guard::cache::ResolveCache*):

```
pub struct VerdictCache {
    table: RwLock<HashMap<String, Entry>>,
    hits: AtomicU64,
    misses: AtomicU64,
    clock: AtomicU64,
    max_entries: usize,
    evict_sample_rate: usize,
}

pub struct ResolveCache {
    table: RwLock<HashMap<String, Entry>>,
    hits: AtomicU64,
    misses: AtomicU64,
    clock: AtomicU64,
    max_entries: usize,
    evict_sample_rate: usize,
}
```

This is a piece of concurrent-data-structure design:

- ***RwLock<HashMap<...>>*** lets any number of workers read a cached verdict concurrently (*read()*), while a write (inserting a new verdict, or evicting) takes the exclusive lock only for the duration of the mutation. Given that verdict lookups usually outnumber insertions, this favours the common case.
- ***AtomicU64* counters (*hits*, *misses*, a logical *clock*) sit outside the lock entirely.** Per-entry *last_used* is also an *AtomicU64*, updated with *Ordering::Relaxed* on every lookup, so recording recency for LRU-ish eviction does not require upgrading a read lock to a write lock.
- **Sampled eviction (*evict_sample_rate*)** avoids scanning the whole table under the write lock when *max_entries* is exceeded: a random sample of entries is drawn via *rand::seq::IteratorRandom* and the least-recently-used of the sample is evicted. This trades eviction precision for bounded lock-hold time regardless of table size.

- **Deny-vs-allow asymmetry** in *ResolveCache*: a *Denied* SSRF verdict is treated as effectively permanent, while *Allowed* verdicts expire after *allow_ttl_ms*.

4.1.3 What is deliberately *not* shared across threads

As ``subresource.rs`` top comment says: "*Bodies are fetched, sanitised and dropped inside the same stack frame, so nothing is shared and no lifetime crosses a thread.*"

Each worker owns its input's entire sub-resource fan-out locally and only the final *Outcome* crosses the channel back to the aggregator.

The *only* state shared between threads is the handful of *Arc*-wrapped, mostly-immutable structures (policy, block-lists, caches), and everything else is worker-local by construction.

This lets *catch_unwind* isolate one worker's panic without any cross-thread cleanup being required.

4.2 Lifetimes

The use of explicit lifetime parameters is concentrated in the hot loop that borrows the input buffer and a handful of policy/cache references for the duration of a single call. Some examples are:

4.2.1 *UrlChecker*<'a>

```
pub struct UrlChecker<'a> {
    blockset: &'a BlockSet,
    skeletons: &'a SkeletonSet,
    verdictcache: &'a VerdictCache,
    rules: &'a UrlRules,
}
```

UrlChecker borrows all four of its dependencies rather than owning *Arc* clones of them.

The *Engine* already owns these structures behind *Arc*, so *UrlChecker* is constructed as a *view* over state that already outlives it.

The lifetime 'a ties the checker's validity to the borrow of the engine's compiled policy artefacts, and the compiler statically guarantees a *UrlChecker* cannot outlive the data it inspects.

4.2.2 *Ctx*<'a> in the HTML rewriter

```
struct Ctx<'a> {
    rules: &'a HtmlRules,
    url: &'a UrlChecker<'a>,
    input: &'a [u8],
    newlines: &'a [usize],
    actions: RefCell<Vec<SanitisationAction>>,
    refused: Cell<bool>,
    references: RefCell<Vec<Reference>>,>
```

```

    base: RefCell<Option<String>>,
}

```

- *rules*, *url*, *input*, and *newlines* are all **borrowed**, because *Ctx* is built once per *sanitize_html* call and dropped at the end of it.
- *actions*, *refused*, *references*, and *base* are **owned but wrapped in *Cell/RefCell***, not borrowed, because they are *written to* by the per-element callback *lol_html* invokes.

The net effect: *Ctx*<'a> cannot leak out of the function that builds it, so there is no risk of an HTML rewrite result referencing a dangling *input* buffer.

4.2.3 Zero-copy parsing, and where it stops

sanitize_html's signature *fn sanitize_html(input: &[u8], rules: &HtmlRules, url: &UrlChecker) -> HtmlOutcome* shows that the input document is never copied into an owned buffer before being parsed.

lol_html tokenizes directly over the borrowed *&[u8]*, and *Ctx* borrows that same slice for the lifetime of the call.

Locations reported in *SanitisationAction* are computed against that same borrowed buffer rather than against a re-scanned copy.

Zero-copy stops, deliberately, at the write side and at the thread boundary:

- **The rewriter's output is necessarily an owned *Vec<u8>*** because sanitisation only sometimes leaves bytes unchanged.
Where it removes a *<script>* block or rewrites a *javascript:* URL, the output diverges from the input.
- **Nothing is shared across threads as a borrowed *Arc<[u8]>* buffer.**
Ctx<'a>'s borrows never have to become *Send* or *Sync*, because they never cross a thread.
Each worker in *process_batch* owns its input's bytes locally, runs the whole acquire → sniff → route → sanitise → report pipeline against that local buffer, and only the final, fully-owned *Outcome* is moved across the *mpsc* result channel.
- **Parser state itself never needs to be *Send*.** Because *Ctx* and the *HtmlRewriter* it drives are constructed, used, and dropped entirely within one call on one thread, the question “can this parser state be handed to another thread mid-parse” never arises.

4.3 Error handling

The project layers three distinct error-handling mechanisms, each used where it fits, rather than picking one uniformly:

4.3.1 Typed errors via *thiserror*

Every module boundary that can fail in a way callers need to distinguish defines its own *#[derive(Error)]* (e.g. *ConfigError* in *policy/mod.rs*).

Each variant carries the structured data needed to act on it, and this is what lets the report layer surface a precise, machine-readable cause rather than an opaque failure.

4.3.2 *anyhow/Box<dyn Error>* at the front-end boundary

main.rs uses *Result<u8, Box<dyn Error>>* for *run()*, and *anyhow* is a listed dependency for the same layer: once an error has crossed from a specific library module into the CLI front-end, its precise type stops mattering, the front-end's only remaining job is to print it and choose an exit code.

This creates a split: typed errors (*thiserror*) inside the library where callers act on variants, erased errors (*anyhow/ Box<dyn Error>*) at the binary boundary.

4.3.3 Panic isolation as a last line of defence

Engine::process_indexed wraps the whole per-input pipeline in *catch_unwind(AssertUnwindSafe(...))*. This is to avoid a bug (an unexpected panic or other) taking down a whole batch run.

Combined with the worker-local ownership, a caught panic in one worker's input cannot corrupt shared state, the *Arc* it read are still valid, and the panic unwound only its own stack frame.

catch_unwind requires its closure to be *UnwindSafe*, a marker trait that says no reference captured by this closure will be observed in a torn, inconsistent state if we unwind through it and keep going.

The engine's per-input state does not actually satisfy this automatically (interior mutability like *RefCell/Cell* is not *UnwindSafe* by default), so the code asserts the safety manually.

This assertion is sound *because* of the worker-local ownership discipline: nothing captured by the closure is shared with another thread that could observe a torn value, so there is no actual unwind-safety hazard to assert away.

4.4 *unsafe* code

unsafe has been entirely avoided for this project. As the project requests said, use of *unsafe* was discouraged, hence the entirety of the project has been made with the idea to avoid *unsafe* code.

Experimental Evaluation

5.1 Correctness

5.1.1 The corpus and ground truth

corpus/benign holds 41 real, unmodified pages pulled from the public web, *corpus/malicious* holds 41 samples (42 files, one of which, *blocklist.txt*, is supporting data rather than a sample) written specifically for this project, mostly by injecting a specific attack technique into the *same* real pages used in the benign set, so a detection result is a same-content, with/without-payload comparison rather than a comparison across unrelated documents.

Every sample is documented in *corpus/ground-truth.toml*: where it came from, its licence, what it's expected to carry, the rule IDs it must trigger (for malicious samples) or is *allowed* to trigger (for benign samples, since a rule can correctly fire even on those), and literal byte markers that must **not** survive sanitisation (*forbidden*) for malicious samples or must survive (*preserved*) for benign samples.

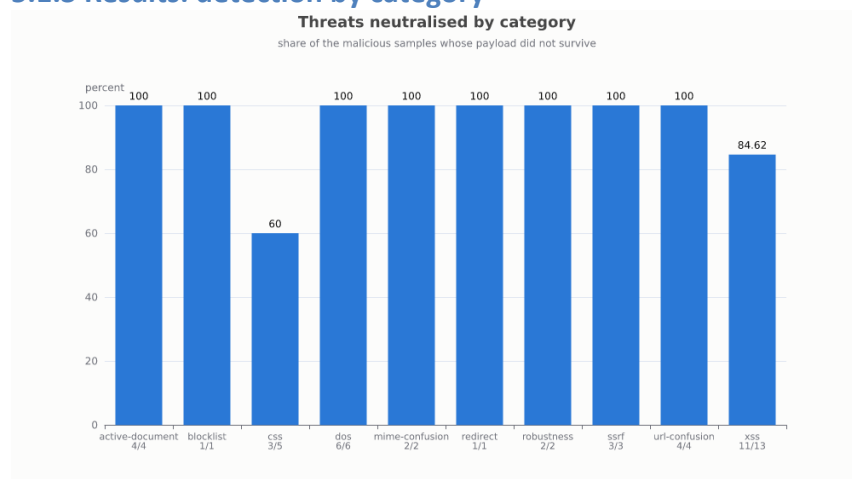
5.1.2 Verdict methodology

xtask::correctness classifies every sample into one of six verdicts, which is meaningfully more precise than a plain pass/fail:

Verdict	Meaning
Detected	(malicious only) the sample reached an accepted status, every required rule fired, and no forbidden byte marker survived in the output
Mislabelled	(malicious only) the attack was neutralised (no leak, accepted status) but not by the rule the ground truth expected
Missed	(malicious only) neutralised but a required rule never fired, and nothing leaked
Leaked	(malicious only) a forbidden byte marker is still present in the sanitised output
Clean	(benign only) no unexpected/forbidden rule fired and no marker that should have survived was lost
FalsePositive	(benign only) either an unjustified rule fired, or content the sample explicitly expected to survive was removed

FalsePositive on the benign side covers *both* directions of over-reach: a rule firing where nothing in that page justifies it, and a rule (correctly, per its own logic) removing something the ground truth says should have been preserved.

5.1.3 Results: detection by category



Threats neutralised by category

Across the 41 malicious samples, **37 were fully neutralised (90.2%)**. Eight of ten attack categories hit 100%: *active-document* (4/4), *blocklist* (1/1), *dos* (6/6), *mime-confusion* (2/2), *redirect* (1/1), *robustness* (2/2), *ssrf* (3/3), and *url-confusion* (4/4).

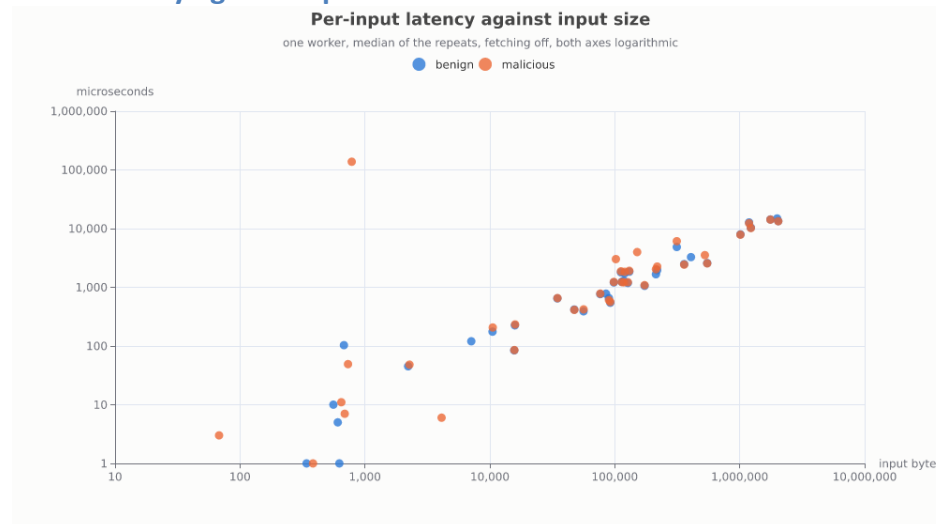
Two categories fell short of 100%:

- **css: 3/5 (60%)** — the weakest category in the corpus.
- **xss: 11/13 (84.6%)** — two samples not fully neutralised, out of the largest single category.

We can state thanks to this that the success rate was **90.2% overall, with css being the weakest section**.

5.2 Performance

5.2.1 Latency against input size



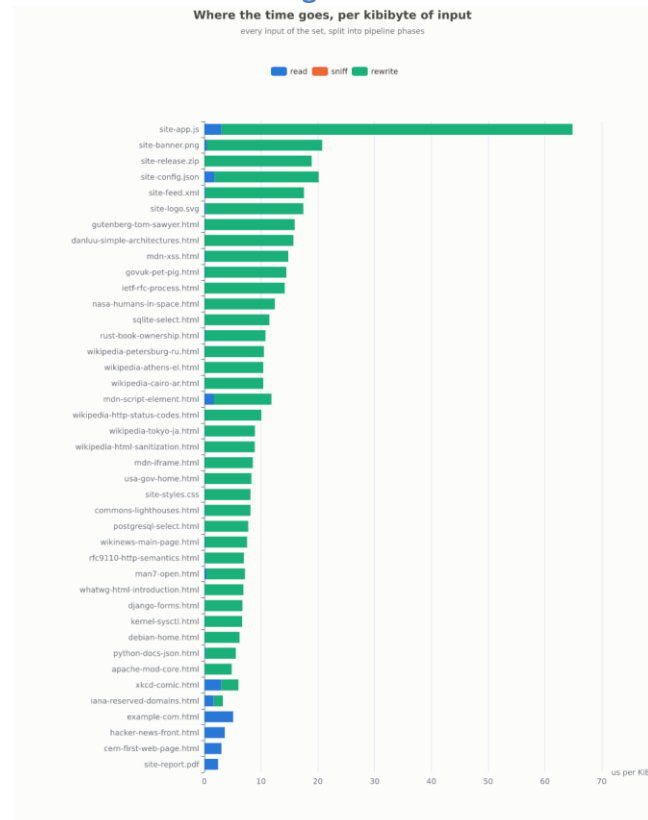
Per-input latency against input size

Measured with *cargo xtask latency*: one worker, median of 5 repeats, fetching off.

The relationship is close to linear on a log-log plot across nearly four orders of magnitude of input size (hundreds of bytes to low megabytes).

Benign and malicious latency track each other closely at matched sizes, with one visible outlier: a malicious sample around 700 bytes takes roughly 130ms.

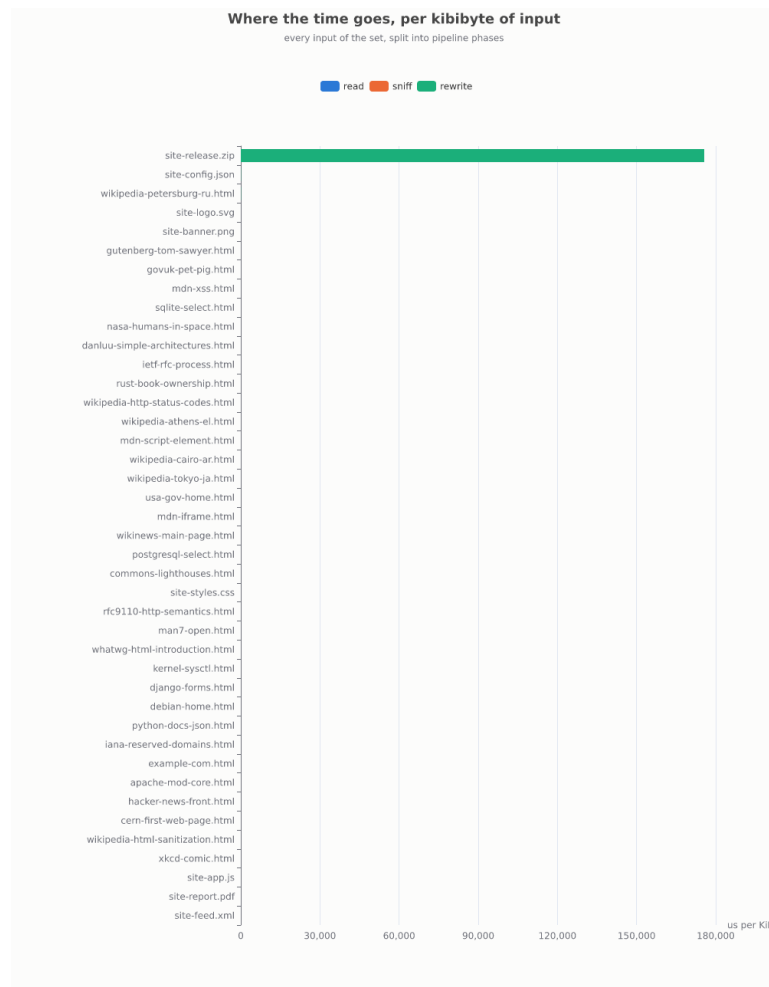
5.2.2 Where the time goes



Phase breakdown, benign corpus

Measured with *cargo xtask phases*: each input split into *read* / *sniff* / *rewrite* phases, normalised to microseconds per kibibyte so inputs of different sizes are comparable. For nearly every benign sample, *rewrite* dominates, with a handful of exceptions where *read* dominates instead: these are small files that the fixed per-call I/O overhead outweighs the amount of actual rewriting work.

site-app.js stands out as the highest per-KiB rewrite cost in the benign set by a wide margin. Since JavaScript sub-resources are routed to the active-content scanner rather than the HTML rewriter, this is consistent with per-byte scanning being more expensive than HTML tag/attribute stripping.

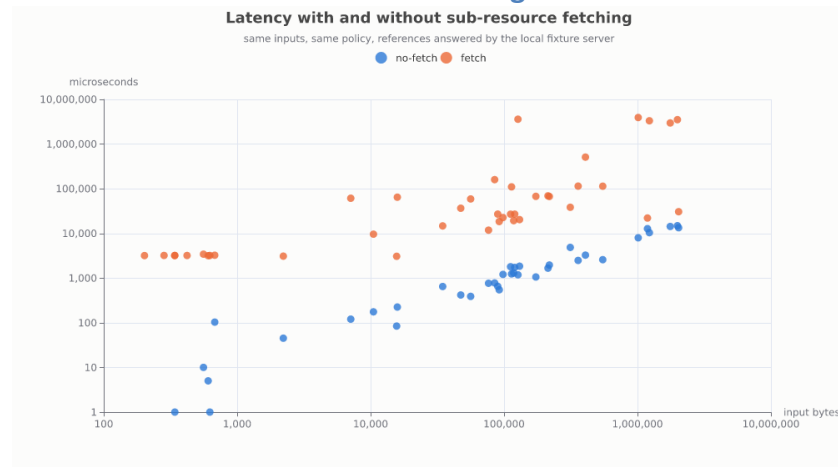


Phase breakdown, malicious corpus

The malicious-corpus version of the same chart is very different: ***site-release.zip* costs roughly 175,000 μ s/KiB, over 1000 times every other sample in the set**, which is otherwise not even visible on the same linear scale.

There's no clear explanation to why the zip takes so long, as it should be immediately refused once the ratio is considered exceeded, but this could also be caused by the cost of reading the central directory, compared to the small size of zip files.

5.2.3 The cost of sub-resource fetching



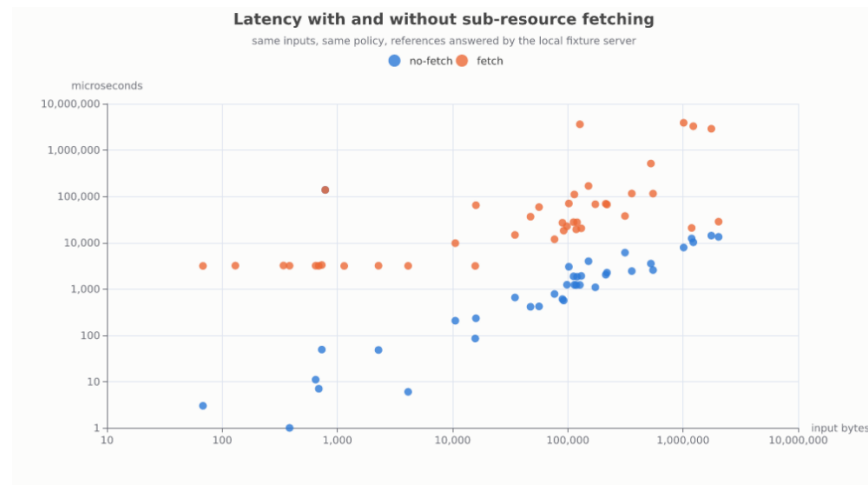
Latency with and without fetching, benign corpus

Every absolute reference in a fetched page is rewritten to point at an in-process fixture server rather than the open web, so the comparison isolates the sub-resource loop's own overhead from real-network variance.

The gap is substantial and consistent across the whole size range: fetch latency sits roughly two to three orders of magnitude above no-fetch latency for the same input.

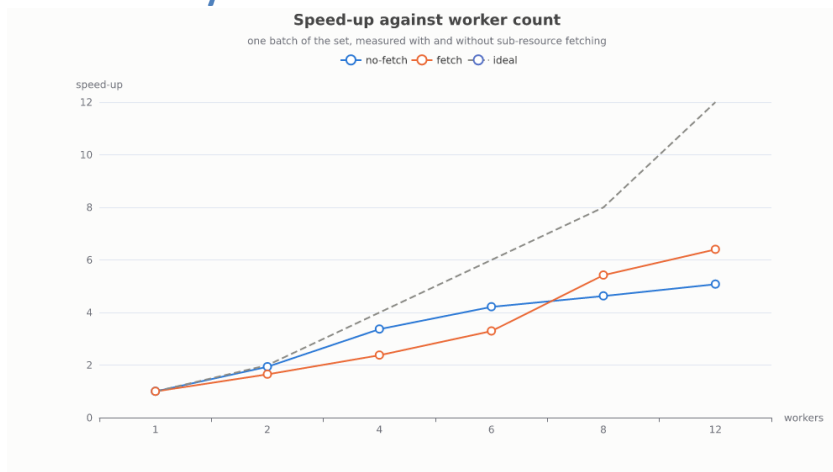
A deployment turning on `subresources.fetch_subresources` should expect multi-millisecond, not sub-millisecond, per-input latency even against a fast, local origin.

The malicious-corpus version of this chart shows the same shape.

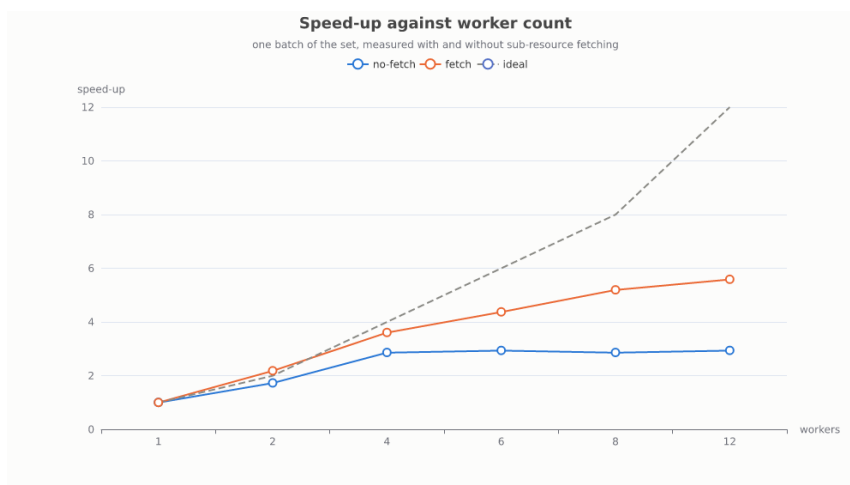


Latency with and without fetching, malicious corpus

5.3 Scalability



Speed-up against worker count, benign corpus



Speed-up against worker count, malicious corpus

This is a benchmark sweeping $WORKERS = [1, 2, 4, 6, 8, 12]$ over three cycles of the whole corpus, run separately for benign/malicious sets and fetch on/off.

This aims to show the impact of multi-threading over the sanitisation.

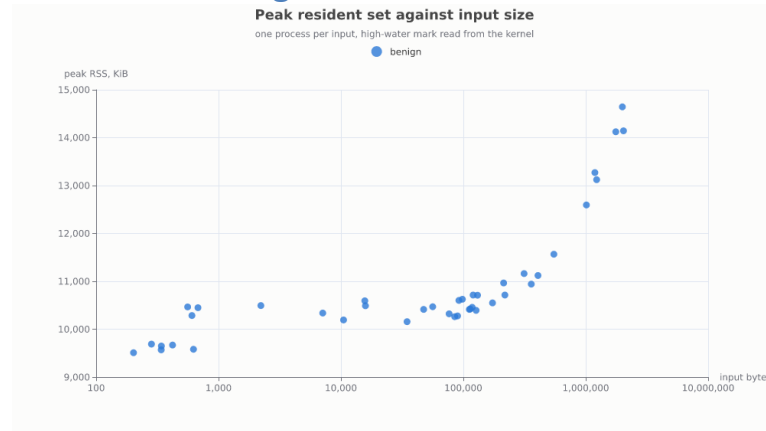
Two things worth noting in these curves:

- **Neither curve tracks the ideal linear line much past 2 workers.**
No-fetch speed-up on the benign set reaches roughly $4.2\times$ at 6 workers and only $5.1\times$ at 12, below the ideal $12\times$.
This might be either because of the *RwLock*-guarded verdict/resolve caches becoming a contention point under enough concurrent workers, or the benchmark running on fewer physical cores than 12.
- **Fetch scales *better* than no-fetch past roughly 6-8 workers, crossing over on both the benign and malicious sets.**
At 12 workers, fetch reaches $\sim 6.3\times$ (benign) and $\sim 5.6\times$ (malicious) speed-up

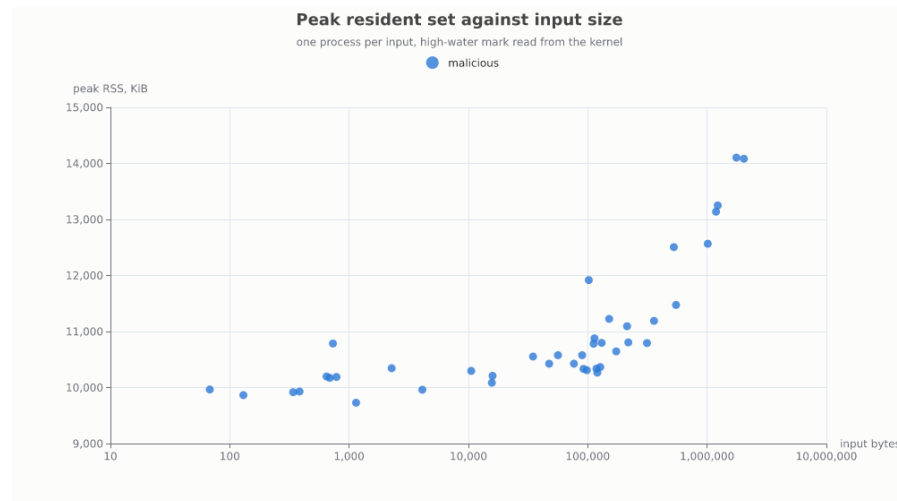
against no-fetch's $\sim 5.1\times$ and $\sim 2.9\times$ respectively.

Fetch-enabled work spends part of its time blocked on I/O, and blocked threads don't compete for CPU the way the no-fetch case's purely CPU-bound rewriting does. This creates a peculiar case where adding workers keeps paying off for the fetch case.

5.4 Resource usage



Peak RSS against input size, benign corpus

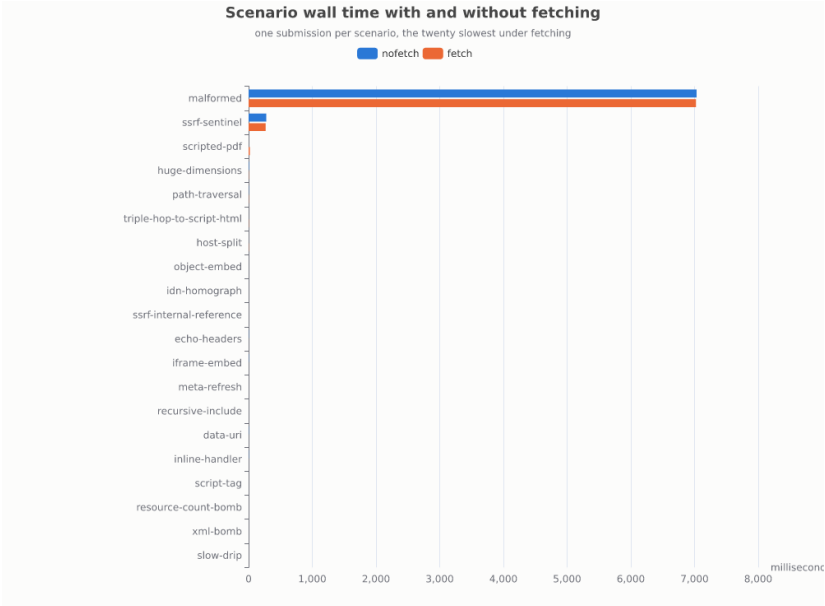


Peak RSS against input size, malicious corpus

Measured with *cargo xtask memory*, which spawns a fresh process per input and reads the kernel's own RSS for that process .

Peak RSS ranges from roughly 9.5 MiB (smallest inputs) to 14.5 MiB (largest, ~ 2 MiB inputs) across both sets. This growth is modest compared against a nearly four-order-of-magnitude size range, and the shape is consistent with a largely fixed per-process baseline.

5.5 The adversarial-scenario harness



Scenario wall time with and without fetching

One result here stands out: **the *malformed* scenario takes roughly 7 seconds in both fetching and non-fetching mode.**

max_time_ms is only ever checked inside the sub-resource-fetch loop: Acquire, Sniff, Route, and Sanitise never consult a deadline themselves.

The rest of the scenarios (aside from *ssrf-sentinel* and the fetch enabled *scripted-pdf*) are not even visible, taking less than 1/10 of a second to process.

The much longer duration of *malformed* is probably due to the sanitiser handling malformed inputs similarly as exceptions, thus incurring into timeouts in order to resolve the sanitisation.

5.6 Summary

Axis	Spec requirement	Result
Correctness	Detection/false-positive rate on a labelled corpus	90.2% overall (37/41); 8/10 categories at 100%, <i>css</i> (60%) and <i>xss</i> (84.6%) below
Performance	Throughput/latency vs. input size	Close to linear on a log-log plot; fetching adds 2-3 orders of magnitude of latency at matched size
Scalability	Speed-up curves vs. worker count	Sub-linear past 2 workers on both sets; fetch scales better than no-fetch past 6-8 workers, plausibly from I/O-wait overlap
Resource usage	Peak memory vs. zero-copy design	9.5-14.6 MiB across the corpus's size range; growth present but modest, and hard to separate from fixed per-process baseline

Critical Assessment: Limitations and Possible Extensions

This chapter's goal is to point out possible weaknesses or limitations of the sanitiser, and also explain possible extensions that could be developed.

6.1 The zip-bomb check trusts declared metadata, not verified decompression

src/scan/dos/zip/mod.rs does not depend on a ZIP-handling crate. It hand-parses the End-Of-Central-Directory record and walks the central directory entries directly, reading each entry's *declared compressed_size* and *uncompressed_size* fields.

ZipBudgets's *max_compression_ratio*, *max_total_uncompressed_bytes*, *max_entry_count* are therefore checked against what the archive's central directory *says* about itself, not against bytes actually produced by inflating it.

This introduces a weakness in case the declared sizes are understated. An attacker with the ability to modify the archive could trick the sanitiser into letting the zip through, despite it carrying malicious content.

Closing this weakness would mean either adopting a streaming-decompression crate that can enforce a ratio cap against bytes actually produced, or cross-checking a sample of entries' true inflated size against their declared size before trusting the metadata for the rest of the archive.

6.2 MIME-type coverage is deliberately limited to widespread formats

The *MimeType* enum lists several formats that are recognised by file extension and by declared *Content-Type* string, but that **do not have a magic-number byte-sniffing rule**: the project's test suite exercises byte-level detection for most of the widespread formats, but there is no equivalent test for, as an example, a RIFF/WEBP FourCC or for the OLE2/Compound File Binary signature that legacy *.doc/.xls/.ppt* files use, or even EMF.

This decision was made to not widen too much the scope of the sanitiser, and keep the focus on the more common vectors of attacks.

6.3 Sub-resource policy is uniform across types, not per-type

SubresourcesRules applies *sniff_rule*, *active_content_rule*, and *dos_risk_rule* as single, global settings across every fetched sub-resource type.

This generalization does not allow for certain types of sub-resources to be allowed while others are refused, as the rules are enforced uniformly across all types.

Extending the policy to handle each sub-resource type differently would increase the effectiveness of the sanitiser, allowing the user to customize subresource handling more in depth. In the current version of the sanitiser this wasn't done both for time reasons and because it was deemed out of scope.

6.4 Active-content rewriting is safe and linear for only four formats

scan/active/mod.rs's *rewrite_if_possible* has a real, format-specific byte-rewriting path for exactly four types, PDF, TIFF, SVG, and CSS, rewriting is applied only when a format-specific rewrite exists, otherwise passes the original bytes through unchanged.

For every other type where active content is *detected* (JavaScript, ZIP/OOXML documents carrying a VBA macro project, XML with an XXE-style construct), there is no corresponding rewrite, under a policy of *ActiveContentAction::Allow*, the detection is recorded as a *SanitisationAction*, but the bytes that pass through are unchanged. Only *ActiveContentAction::Reject* actually stops the content from reaching the output for these types.

The reason is: PDF, TIFF, and SVG admit a linear, well-bounded way to strip the dangerous construct without disturbing the rest of the file's structure, and CSS is tokenizable cleanly enough for the same treatment.

Office formats with macros are a different problem: a *.docx/.xlsx/.pptx* file is a container whose internal structure is considerably more interdependent than an image or a stylesheet, and surgically removing active content risks leaving the rest of the archive in a state that no longer opens correctly in the target application.

Due to the way more complex action of rewriting in these other formats, we preferred directly refusing active content where rewriting proved too difficult and risked corrupting the file.

6.5 A development-environment limitation: Docker networking on Windows

This one is not a limitation of the sanitiser itself but of running its evaluation tooling on Windows, and it is documented in the README in some detail.

On Windows, the scenario suite can take roughly 3.3 seconds per HTTP connection, enough that a redirect chain pays that cost once per hop, and the *triple-hop-to-script-html* scenario alone takes 13 seconds.

Three things combine to produce it:

1. *localhost* resolves to *::1* (IPv6) before *127.0.0.1* (IPv4) on Windows.
2. With Docker Desktop's *networkingMode=mirrored* set, a published port is claimed on both address families but only actually served over IPv4, and a connection attempt to *[::1]:<port>* is silently black-holed rather than refused, so the client waits out a full connect timeout instead of failing over immediately.
3. *ureq* tries resolved addresses in sequence and splits its connect budget geometrically across them, with two candidate addresses, the dead *::1* consumes roughly two-thirds of the configured *connect_timeout_ms* before the client moves on to the working *127.0.0.1* address.

The documented fix is to leave *networkingMode* unset in *%USERPROFILE%\wslconfig* rather than to change anything in the sanitiser or its policy. As a safety net, it's also recommended to additionally set a short *connect_timeout_ms = 450*.

Linux and macOS are unaffected.

6.6 Possible extensions

Two extensions have been discussed:

1. **Package-manager distribution.**

The project already has a *cargo-dist*-based release pipeline that builds platform-specific archives for five targets and publishes them to GitHub Releases automatically on a version tag, but *dist-workspace.toml*'s *installers* list is currently empty, so what ships is a set of raw *.tar.gz/.zip* archives a user must fetch and unpack by hand.

Publishing to *apt* and Homebrew would make the tool installable the way most comparable CLI security tools are, without requiring a Rust toolchain on the user's machine.

2. **A reusable GitHub Action for CI-scanner embedding.**

The original specification frames one use case as embedding into CI scanners, and the tool's CLI/library split already supports that use case mechanically, but there is currently no published, reusable GitHub Action wrapping it.

Packaging the CLI as a composite or Docker-based Action would let another project's CI pipeline run this sanitiser as a scanning step without building it themselves.